

Contribute to Real ML Software

A practical open-source path from learner → debugger → contributor → ML engineer.

Every repository gives you three direct doors

- ① Repository — understand the codebase
- ② Issues — find real problems
- ③ Contribution Guide — learn the rules before touching code

Then enter the engineering loop

Read → Run → Reproduce → Investigate → Hypothesize → Patch → Test → Review → PR

Do not treat the issue list as homework.

Issue status changes. Always open the live issue, read recent discussion, check linked PRs, and confirm that maintainers are accepting work before investing time.

Publication note: Current issue examples in this document were checked against GitHub during generation on 6 September 2026. Re-check before publishing a learner-facing edition.

Your Open-Source Progression

The repository becomes progressively less like a tutorial and more like a real engineering environment.

01 READ	Map the repository, contribution guide, tests, architecture and issue conventions.
02 RUN	Set it up locally. Make the existing tests pass before changing anything.
03 REPRODUCE	Turn an issue into a deterministic local reproduction or regression test.
04 INVESTIGATE	Trace the behavior through the codebase and form a root-cause hypothesis.
05 PATCH	Make the smallest change that addresses the actual cause.
06 VALIDATE	Run targeted tests, relevant broader tests and benchmarks when needed.
07 REVIEW	Read your diff like a maintainer. Remove unrelated changes.
08 PR	Explain the problem, root cause, fix, tests and trade-offs.
09 TRANSFER	Repeat the loop in a different repository with less mentor scaffolding.

The standard: never submit code you cannot explain. AI can help you search, reason and debug, but your contribution must be reproducible, understandable and defensible.

01 scikit-learn

Classical ML • statistics • APIs • testing

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■
ENGINEERING FOCUS Docs + ML reasoning	
What this teaches: Classical ML • statistics • APIs • testing.	
How to contribute: Issue #34807 — Ridge solver documentation does not clearly explain when a dual formulation is used.	
Open the referenced issue → Verify status and recent maintainer discussion before starting.	

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

02 NumPy

Numerical computing • arrays • performance • native code

REPOSITORY	Open repository ■
------------	-----------------------------------

ISSUE TRACKER	Open issue tracker ■
---------------	--------------------------------------

CONTRIBUTION GUIDE	Open contribution guide ■
--------------------	---

ENGINEERING FOCUS Advanced debugging
What this teaches: Numerical computing • arrays • performance • native code.
How to contribute: Issue #32488 — deeply nested structured dtypes can cause a native SIGSEGV during array construction.

Open the referenced issue → Verify status and recent maintainer discussion before starting.
--

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

03 pandas

Data manipulation • I/O • validation • performance

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■
ENGINEERING FOCUS Data engineering	
What this teaches: Data manipulation • I/O • validation • performance.	
How to contribute: Issue #68026 — <code>to_xml()</code> bool output uses capitalized True/False and is reported as invalid for <code>xs:boolean</code> .	
Open the referenced issue → Verify status and recent maintainer discussion before starting.	

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

04 XGBoost

Boosting • model serialization • cross-platform systems

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■
ENGINEERING FOCUS ML systems	
What this teaches: Boosting • model serialization • cross-platform systems.	
How to contribute: Issue #12459 — a Linux-trained Booster can fail to unpickle on Windows with the same XGBoost version.	
Open the referenced issue → Verify status and recent maintainer discussion before starting.	

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

05 PyTorch

Deep learning • autograd • GPU • distributed systems

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■

ENGINEERING FOCUS Deep learning → systems
What this teaches: Deep learning • autograd • GPU • distributed systems.
How to contribute: Use the live issue tracker to select a reproducible, appropriately scoped framework issue; avoid jumping straight into kernel/distributed work.

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

06 Hugging Face Transformers

Transformers • LLMs • generation • inference

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■

ENGINEERING FOCUS Modern AI
What this teaches: Transformers • LLMs • generation • inference.
How to contribute: Choose a current issue involving generation, model integration, tokenization or docs that you can reproduce and explain.

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

07 MLflow

Experiment tracking • model lifecycle • tracing • MLOps

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■
ENGINEERING FOCUS MLOps	
What this teaches: Experiment tracking • model lifecycle • tracing • MLOps.	
How to contribute: Issue #25551 — create_experiment can leave a dangling experiment when trace-location linking fails.	
Open the referenced issue → Verify status and recent maintainer discussion before starting.	

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

08 JupyterLab

Developer tooling • testing • notebooks • large codebase

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■
ENGINEERING FOCUS Software engineering	
What this teaches: Developer tooling • testing • notebooks • large codebase.	
How to contribute: Issue #19002 — proposal to encode practices that reduce flaky UI tests as linter rules.	
Open the referenced issue → Verify status and recent maintainer discussion before starting.	

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

09 TensorFlow

Deep learning framework • APIs • distributed execution

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■

ENGINEERING FOCUS Deep learning → systems
What this teaches: Deep learning framework • APIs • distributed execution.
How to contribute: Use the current tracker to find a contained bug, test, documentation or API issue that can be reproduced locally.

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

10 Ray

Distributed computing • training • serving • scheduling

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■

ENGINEERING FOCUS Distributed ML
What this teaches: Distributed computing • training • serving • scheduling.
How to contribute: Select a distributed execution or serving issue where you can build a minimal reproduction before attempting larger cluster problems.

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

11 DVC

Data/model versioning • reproducibility • pipelines

REPOSITORY	Open repository ■
ISSUE TRACKER	Open issue tracker ■
CONTRIBUTION GUIDE	Open contribution guide ■

ENGINEERING FOCUS MLOps → reproducibility
What this teaches: Data/model versioning • reproducibility • pipelines.
How to contribute: Choose a reproducibility, pipeline or storage-integration issue and turn it into a minimal regression test.

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

12 Kubeflow

Kubernetes • ML platforms • pipelines • deployment

REPOSITORY	Open repository ■
------------	-----------------------------------

ISSUE TRACKER	Open issue tracker ■
---------------	--------------------------------------

CONTRIBUTION GUIDE	Open contribution guide ■
--------------------	---

ENGINEERING FOCUS Platform engineering
What this teaches: Kubernetes • ML platforms • pipelines • deployment.
How to contribute: Start with a contained docs, UI or component issue; progress toward platform-level work only after you can run the relevant component.

Mentor move → Give these three links to your AI mentor. Ask it to inspect the contribution guide first, then choose one issue matching your current capability. The mentor should not give you the patch; it should guide your investigation.

The Final Open-Source Proof

You are ready to call yourself an ML engineer when you can enter unfamiliar software and create value without a tutorial holding your hand.

UNDERSTAND	Explain the relevant architecture and why the affected code exists.
REPRODUCE	Produce a minimal, deterministic reproduction.
ROOT CAUSE	Explain the causal path, not just the changed line.
IMPLEMENT	Make a focused change consistent with project conventions.
TEST	Add or update regression coverage and run relevant checks.
REVIEW	Critically review your own diff and respond to maintainer feedback.
COMMUNICATE	Write a clear issue/PR describing problem, cause, fix and evidence.
TRANSFER	Repeat the entire loop in another unfamiliar ML repository.

TRUE ML ENGINEER SIGNAL

You can learn an unfamiliar codebase, find a real problem, investigate it, make a defensible change, prove that the change works, communicate with maintainers, and repeat the process somewhere new.